

College ERP System (Campuzz)

Complete Technical Cheat Sheet & Viva Defense Guide

This study guide contains a detailed breakdown of all **12 core technical topics**, frameworks, libraries, background services, and architecture concepts implemented in this project. Use this guide to confidently explain *where*, *what*, and *how* each technology is used, along with code syntax and answers to potential viva questions.

1. Baileys WhatsApp Integration (@whiskeysockets/baileys)

■ **Where Used in Project:** frontend/whatsappService.js, frontend/vite.config.js, frontend/baileys_auth_info_*

■ **What It Is:** Node.js multi-device library executing WhatsApp Web WebSocket protocol directly without requiring official Meta API keys or Selenium browsers.

■ **How It Is Used:** Implements a DualWhatsAppRouter class controlling two distinct WhatsApp accounts (Faculty Channel & Student Channel). Uses useMultiFileAuthState to serialize encryption keys to disk so re-authentication via QR code is not needed on server restarts. Configured as server middleware inside Vite's dev server.

■ Basic Code Snippet:

```
import makeWASocket, { useMultiFileAuthState } from '@whiskeysockets/baileys';

const { state, saveCreds } = await useMultiFileAuthState('./auth_dir');
const sock = makeWASocket({ auth: state });
sock.ev.on('creds.update', saveCreds);

// Send message to WhatsApp JID format
const jid = '918866541102@s.whatsapp.net';
await sock.sendMessage(jid, { text: 'Attendance Alert: Absent in Lecture 2' });
```

■ Viva / Professor Question Defense:

Q: How does Baileys send WhatsApp messages without official API fees?

→ Baileys emulates the WhatsApp Web multi-device Noise protocol. It authenticates as a paired web browser client using WebSocket frames.

Q: What is a JID format?

→ JID stands for Jabber ID (from XMPP protocol). Individual WhatsApp accounts are formatted as [CountryCode][Number]@s.whatsapp.net.

Q: How does authentication state persist across restarts?

→ useMultiFileAuthState stores cryptographic keys and session tokens inside JSON files in baileys_auth_info_* folders.

2. React 19 & Component Architecture

■ **Where Used in Project:** frontend/src/App.jsx, AdminDashboard.jsx, FacultyDashboard.jsx, StudentDashboard.jsx, Landing.jsx

■ **What It Is:** Declarative component-based UI library with Single Page Application (SPA) architecture.

■ **How It Is Used:** Follows role-based dynamic view rendering. App.jsx checks user session state (admin, faculty, student) and dynamically mounts the corresponding dashboard component without reloading the page.

■ Basic Code Snippet:

```
// App.jsx - Dynamic Role Router
function App() {
  const [user, setUser] = useState(null);
  if (!user) return <Landing onLogin={setUser} />;
  if (user.role === 'admin') return <AdminDashboard user={user} />;
  if (user.role === 'faculty') return <FacultyDashboard user={user} />;
  return <StudentDashboard user={user} />;
}
```

■ Viva / Professor Question Defense:

Q: What is JSX and how does the browser execute it?

→ JSX (JavaScript XML) allows HTML markup inside JS code. Vite uses Babel/esbuild to convert JSX into standard React.createElement() function calls before serving.

Q: What is the Virtual DOM?

→ An in-memory lightweight representation of the real DOM. React computes differences (reconciliation) and updates only changed nodes in the real browser DOM.

3. React Hooks (useState, useEffect, useRef)

■ **Where Used in Project:** All component files across frontend/src/

■ **What It Is:** Built-in React functions for managing component state, side effects, and persistent DOM references.

■ **How It Is Used:** useState stores active tab, form data, loading spinners, and API responses. useEffect executes API data fetching on component mount or state changes. useRef maintains non-rendering references for Canvas/SVG elements.

■ Basic Code Snippet:

```
const [students, setStudents] = useState([]);
const [loading, setLoading] = useState(true);

useEffect(() => {
  let isMounted = true;
  apiService.get('/api/students/').then(data => {
    if (isMounted) {
      setStudents(data);
      setLoading(false);
    }
  });
  return () => { isMounted = false; }; // Cleanup function
}, []); // Empty array = run once on mount
```

■ Viva / Professor Question Defense:

Q: What happens if useEffect has an empty dependency array []?

→ The effect callback runs exactly once after initial component mounting (equivalent to componentDidMount).

Q: What is the cleanup function inside useEffect?

→ The return function inside useEffect runs before unmounting or re-execution, preventing memory leaks by canceling pending requests or listeners.

4. Vite Dev Server, Express-style Middleware & Reverse Proxy

■ **Where Used in Project:** frontend/vite.config.js

■ **What It Is:** Next-generation frontend build tool with dev server middleware extension and proxying capabilities.

■ **How It Is Used:** Uses Vite's configureServer(server) hook to mount custom Node.js middleware for /api/send-whatsapp and /api/send-email endpoints. Defines server.proxy to forward all database REST calls to Django on port 8000.

■ Basic Code Snippet:

```
export default defineConfig({
  plugins: [
    react(),
    {
      name: 'api-server',
      configureServer(server) {
        server.middlewares.use(async (req, res, next) => {
          if (req.url === '/api/send-whatsapp' && req.method === 'POST') {
            // Custom Node endpoint handling
          } else { next(); }
        });
      },
    },
  ],
  server: { proxy: { '/api': 'http://localhost:8000' } }
});
```

■ Viva / Professor Question Defense:

Q: Why do we use Vite proxy in development?

→ To bypass browser **CORS (Cross-Origin Resource Sharing)** restrictions between frontend (port 5173) and backend (port 8000) during local development.

5. Automated Email Service (Nodemailer)

■ **Where Used in Project:** frontend/vite.config.js (Endpoint: /api/send-email)

■ **What It Is:** Node.js SMTP email dispatcher module.

■ **How It Is Used:** Constructs HTML email messages for low attendance alerts and exam notices. Uses Gmail SMTP if credentials exist in .env; otherwise automatically creates an Ethereum test account with preview link fallback.

■ Basic Code Snippet:

```
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransport({
  service: 'gmail',
  auth: { user: 'admin@college.edu', pass: 'app-password' }
});

await transporter.sendMail({
  from: 'admin@college.edu',
  to: 'student@gmail.com',
  subject: 'Attendance Warning',
  html: '<h2>Attendance Alert</h2><p>Your attendance is below 75%.</p>'
});
```

■ Viva / Professor Question Defense:

Q: Why run Nodemailer on the server side instead of inside React?

→ Browsers cannot open raw TCP/SMTP connections due to security sandboxing. Email transmission requires server-side Node.js environment.

6. Axios / REST API Client Abstraction (apiService.js)

■ **Where Used in Project:** frontend/src/apiService.js

■ **What It Is:** Centralized API communication layer wrapping standard Fetch API.

■ **How It Is Used:** Provides clean get(), post(), put(), delete() helper methods. Automatically reads JWT access tokens from localStorage and attaches Authorization: Bearer [token] header to requests.

■ Basic Code Snippet:

```
const getHeaders = () => {
  const token = localStorage.getItem('token');
  return {
    'Content-Type': 'application/json',
    ...(token && { 'Authorization': `Bearer ${token}` })
  };
};

export const apiService = {
  get: async (url) => {
    const res = await fetch(url, { headers: getHeaders() });
    return res.json();
  }
};
```

■ Viva / Professor Question Defense:

Q: How does the app authenticate HTTP requests?

→ By injecting the stored JWT token into the HTTP Authorization request header using the Bearer scheme.

7. Django Backend & MVT Architecture

■ **Where Used in Project:** backend/erp_backend/ (settings.py, urls.py, manage.py)

■ **What It Is:** High-level Python web framework utilizing Model-View-Template (MVT) architecture for REST APIs.

■ **How It Is Used:** Serves as core backend server. settings.py registers sub-apps (users, students, faculty, attendance, marks, notices, predictions, assignments, timetable) and configures CORS/JWT.

■ Basic Code Snippet:

```
# erp_backend/urls.py
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/users/', include('users.urls')),
    path('api/students/', include('students.urls')),
    path('api/attendance/', include('attendance.urls')),
]
```

■ Viva / Professor Question Defense:

Q: What is MVT architecture in Django?

→ Model (Database Data), View (Business Logic/JSON API response), Template (HTML layout / React frontend in our system).

8. Django REST Framework (DRF) & API Serializers

■ **Where Used in Project:** Across all backend sub-apps in serializers.py and views.py

■ **What It Is:** Toolkit for building RESTful web APIs on top of Django.

■ **How It Is Used:** ModelSerializer translates Python ORM model objects into JSON strings and validates incoming HTTP payloads. APIView handles HTTP GET/POST/PUT/DELETE requests.

■ Basic Code Snippet:

```
# serializers.py
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'

# views.py
class StudentList(APIView):
    def get(self, request):
        students = Student.objects.all()
        serializer = StudentSerializer(students, many=True)
        return Response(serializer.data)
```

■ Viva / Professor Question Defense:

Q: What is the main job of a DRF Serializer?

→ To convert complex Python/Django ORM data types into native Python data structures that can easily be rendered into JSON/XML, and vice versa.

9. JWT Authentication & Role-Based Access Control (RBAC)

■ **Where Used in Project:** backend/users/models.py, djangoestframework_simplejwt

■ **What It Is:** Stateless user authentication standard paired with role authorization.

■ **How It Is Used:** Custom User model extends AbstractUser with a role field (admin, faculty, student). SimpleJWT's TokenObtainPairView verifies credentials and returns Access & Refresh tokens carrying encoded user payload.

■ Basic Code Snippet:

```
class User(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('faculty', 'Faculty'),
        ('student', 'Student'),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='student')
```

■ Viva / Professor Question Defense:

Q: What are the three parts of a JWT token?

→ 1. Header (Algorithm & type), 2. Payload (Claims like user_id, role, expiry), 3. Signature (HMAC-SHA256 verification hash).

Q: Why JWT over Django Session cookies?

→ JWT is stateless, eliminating server-side session memory storage and enabling seamless multi-server scalability.

10. Django ORM, Database Schema & Migrations

■ **Where Used in Project:** backend/*/models.py, backend/db.sqlite3

■ **What It Is:** Object-Relational Mapping abstraction layer for SQL database interaction.

■ **How It Is Used:** Defines relational models: User 1-to-1 -> Student/Faculty profile, Student 1-to-Many -> Attendance/Mark records using ForeignKey and on_delete=models.CASCADE.

■ Basic Code Snippet:

```
class Student(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    enrollment_no = models.CharField(max_length=20, unique=True)

class Attendance(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    date = models.DateField()
    is_present = models.BooleanField(default=True)
```

■ Viva / Professor Question Defense:

Q: What is the difference between makemigrations and migrate?

→ makemigrations generates Python migration blueprint scripts from models.py changes. migrate executes SQL statements against SQLite DB.

11. Machine Learning & Performance Predictions (Scikit-Learn)

■ **Where Used in Project:** backend/predictions/ (train_model.py, views.py, performance_model.pkl)

■ **What It Is:** Supervised AI model predicting student GPA/SPI outcomes and risk levels.

■ **How It Is Used:** Trained on historic student metrics (attendance %, mid-sem marks, assignment completion). Uses joblib to serialize trained Decision Tree / Linear Regression model to .pkl file. API view loads model at runtime to predict SPI.

■ Basic Code Snippet:

```
import joblib
from sklearn.tree import DecisionTreeRegressor

# Training & saving model
model = DecisionTreeRegressor(max_depth=5)
model.fit(X_train, y_train)
joblib.dump(model, 'ml_model/performance_model.pkl')

# Runtime Inference API
loaded_model = joblib.load('ml_model/performance_model.pkl')
predicted_spi = loaded_model.predict([[attendance_pct, mid_sem_score, assignment_pct]])
```

■ Viva / Professor Question Defense:

Q: Why use joblib for saving machine learning models?

→ joblib is optimized for serializing Python objects containing large NumPy data arrays, offering faster read/write speeds than default pickle.

12. Dynamic SVG Animation Component (GrowingPlant.jsx)

■ **Where Used in Project:** frontend/src/GrowingPlant.jsx

■ **What It Is:** Reactive vector graphics component visualizing student growth.

■ **How It Is Used:** Dynamically calculates SVG <path> stem height, leaf count, and branch curves based on the student's progress percentage prop.

■ Basic Code Snippet:

```
export function GrowingPlant({ progressPercentage = 75 }) {  
  const stemHeight = (progressPercentage / 100) * 180;  
  return (  
    <svg viewBox="0 0 200 300">  
      <line x1="20" y1="280" x2="180" y2="280" stroke="#8d6e63" strokeWidth="4" />  
      <path d={`M 100 280 Q 95 ${280 - stemHeight / 2} 100 ${280 - stemHeight}`} stroke="#4caf50"  
        strokeWidth="6" fill="none" />  
    </svg>  
  );  
}
```

■ Viva / Professor Question Defense:

Q: Why use inline SVG over static images for visual charts?

→ Inline SVGs allow dynamic DOM manipulation, smooth resolution-independent rendering at any screen size, and low bandwidth footprint.

Quick Reference Summary Table

Topic / Feature	Technology	Primary File	Key Concept to Remember
WhatsApp Messaging	@whiskeysockets/baileys	whatsappService.js	WebSocket Noise Protocol, JID, auth session persistence
Email Notifications	Nodemailer	vite.config.js	SMTP Transport, Ethereal test preview account
Frontend Engine	React 19 + Vite	App.jsx, vite.config.js	SPA, Virtual DOM, Dev Server Proxy middleware
React Hooks	React	Dashboard files	useState (state), useEffect (lifecycle), useRef (DOM/Canvas)
API Client Layer	Fetch API / Axios	apiService.js	Authorization Bearer header, JWT token injection
Backend Framework	Django 6 + DRF	erp_backend/settings.py	MVT Architecture, REST API endpoints, ModelSerializer
Authentication & RBAC	SimpleJWT	users/models.py	Stateless JWT tokens, Role choices (admin/faculty/student)
Database & ORM	Django ORM + SQLite	*/models.py	ForeignKey, OneToOneField, makemigrations, migrate
Machine Learning	Scikit-Learn + Joblib	predictions/views.py	Decision Tree Regressor, .pkl model serialization
Dynamic Charts	Custom SVG	GrowingPlant.jsx	Dynamic SVG path calculation based on progress %